PHYS7101: NUMERICAL METHODS AND DATA ANALYSIS · LECTURE 4

Between the points

Interpolation, extrapolation, numerical differentiation and integration

Mathew Smith & Aneta Stefanovska

Where we left off

From Lecture 3

- Root finding: bisection, Newton
- Optimisation: minima among the roots of ∇f
- Fitting: minimise the misfit to noisy data
- Goodness of fit: residuals, χ^2/ν
- Testing a method where the answer is known

Today

- Interpolation: values between the points
- Extrapolation: values beyond them
- Differentiation: slopes from values
- Integration: areas from values
- One measured-like example throughout Parts C and D

All four start from the same thing: values $f_i = f(x_i)$ at discrete points x_i, with spacing h.

One idea, four tasks

Task	Question	Approach
Interpolation	$f(x)$ between the points?	Pass a simple function through the points
Extrapolation	$f(x)$ outside the points?	The same, but nothing in the data checks it
Differentiation	$f'(x)$ from the values?	Differentiate it: finite differences
Integration	$\int f\,dx$ from the values?	Integrate it: trapezoid, Simpson

The common thread

The simple function is usually a low-degree polynomial on each interval. For interpolation, differentiation and integration its error behaves as Ch^p — the same form as for time stepping in Lecture 2, and checked the same way: halve h and compare.

PART A

Interpolation

Values between the points.

Interpolation is not fitting

	Interpolation (today)	**Fitting** (Lecture 3)
The curve passes	through every point exactly	between the points
Suited to	exact values: formulas, tables, simulation output	values with measurement errors
Number of parameters	equal to the number of points	far fewer than the points

Consequence

Interpolating values with measurement errors reproduces the errors exactly. For measurements, fit first (Lecture 3); interpolate exact values.

Polynomial interpolation

Through $n+1$ points with distinct x_i there is **exactly one** polynomial p of degree $\le n$ with $p(x_i) = f_i$. Lagrange form:

$$p(x) = \sum_{i=0}^{n} f_i \, \ell_i(x), \qquad \ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j} \quad (\ell_i = 1 \text{ at } x_i,\ 0 \text{ at the other points}).$$

If f has $n+1$ continuous derivatives, the error at x is

$$\boxed{f(x) - p(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i)}$$

for some ξ between the points and x.

Two things set the error

How smooth f is (the derivative), and where the points are (the product). The product is small between neighbouring points and grows fast outside them.

The simplest case: linear interpolation

Between neighbours x_i and $x_{i+1} = x_i + h$:

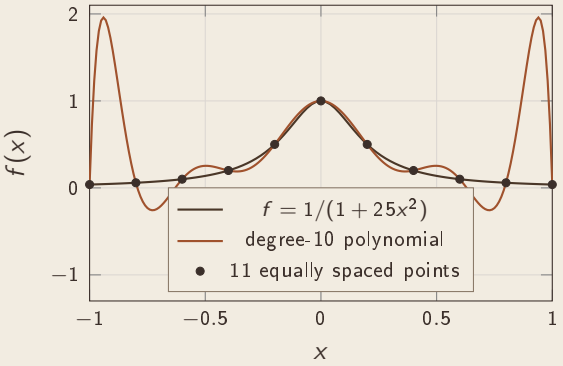
$$p(x) = f_i + \frac{f_{i+1} - f_i}{h}\,(x - x_i), \qquad \boxed{|f(x) - p(x)| \le \frac{h^2}{8} \max |f''|}$$

$f = \sin x$ on $[0, \pi]$, $\max \lvert f'' \rvert = 1$		
h	max error	bound $h^2/8$
0.5	0.0306	0.0313
0.25	0.0078	0.0078
0.125	0.0020	0.0020

Read the table

Halving h divides the error by 4: **order 2**. The bound is almost reached, because $|f''| = 1$ at the top of the sine.

More points can make it worse: Runge's example



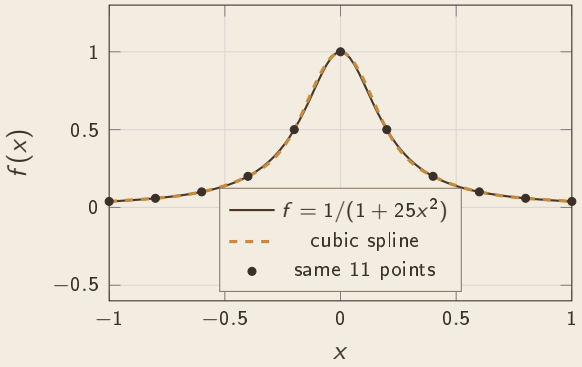
Largest error on $[-1, 1]$:

One polynomial through	max error
11 equally spaced points	1.9
21 equally spaced points	60
11 Chebyshev points	0.11

Lesson

A high-degree polynomial through equally spaced points oscillates near the ends. Chebyshev points, $x_k = \cos\big((2k+1)\pi/22\big)$, $k = 0, \ldots, 10$, crowd towards the ends and tame it.

Cubic splines: low degree, many pieces



One cubic on each interval, joined so that the value, slope and curvature are continuous. Error $O(h^4)$ for smooth f.

Same data	max error
Spline, 11 points	0.022
Spline, 21 points	0.0032

```
scipy.interpolate.CubicSpline(xi, fi)
```

Choosing an interpolant

Method	Error	Use when
Linear	$O(h^2)$	Rough or densely sampled data; never overshoots
Cubic spline	$O(h^4)$	Smooth data: the usual default
One polynomial	—	Few points, or Chebyshev points; never high degree on equally spaced points

A check without the exact answer

Leave out every second point, interpolate from the rest, and compare at the points you left out. The differences estimate the error — the $h/2$ test for interpolation.

Interpolating a time series changes its dynamics

Samples per period	Linear: amplitude loss	Linear: phase error	Cubic spline: amplitude / phase
4	29%	4°	2.8% / 0.6°
8	7.6%	0.5°	0.1% / 0.02°
16	1.9%	0.06°	negligible

Interpolation **flattens fast oscillations**: peaks between samples are cut off. Phases change through common practices:

- **Holding the last value** delays the whole signal by half a sample (22.5° of phase at 8 samples per period)
- **Interpolating a wrapped phase** across a jump from π to $-\pi$: a spurious excursion — unwrap first
- **Resampling irregular data** (e.g. heartbeat times): periods, phase relations and coupling can change

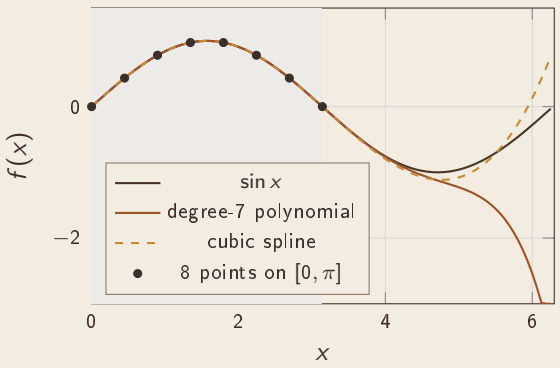
Rule: $\gtrsim 10$ samples per period of the fastest relevant oscillation; the same method for all signals you compare; check with two interpolants.

PART B

Extrapolation

Values beyond the points — where nothing checks the answer.

Outside the data the error grows fast



Both interpolants are excellent inside $[0, \pi]$ (shaded).

x	$\sin x$	polynomial	spline
3.5	-0.35	-0.35	-0.36
5.0	-0.96	-1.23	-1.08
6.0	-0.28	-2.54	$+0.13$

Why

The product $\prod(x - x_i)$ in the error formula grows like a high power of the distance from the data.

Extrapolate with a model, not with a polynomial

When extrapolation is justified

When physics gives the form of f — exponential decay, an orbit, a steady state — fit that model (Lecture 3) and extrapolate the **model**, with its uncertainty.

Even then

The uncertainty grows with distance from the data, and the model holds only while the system does not change — a nonautonomous system (Lecture 2) may not.

Say that you are extrapolating, and on what assumption.

Padding is extrapolation

Any operation on a window of neighbouring values — smoothing, filtering, differences — needs values **beyond the ends**. Padding supplies them by assumption.

21-point moving average of exact samples of $\cos t$ on $[0, 4]$, $\Delta t = 0.05$

Padding	**Assumption beyond the end**	**Error at $t = 0$**	**Error at $t = 4$**
Zeros	$x = 0$	0.45	0.20
Last value repeated	x constant	0.02	0.11
Mirror image	x symmetric about the end	0.00	0.19
Periodic	the record repeats	0.83	0.66

Rule

Each padding is right only where its assumption holds (the mirror: exact at $t = 0$, wrong at $t = 4$). Trim or mark the results within half a window of each end.

PART C

Numerical differentiation

Slopes from values.

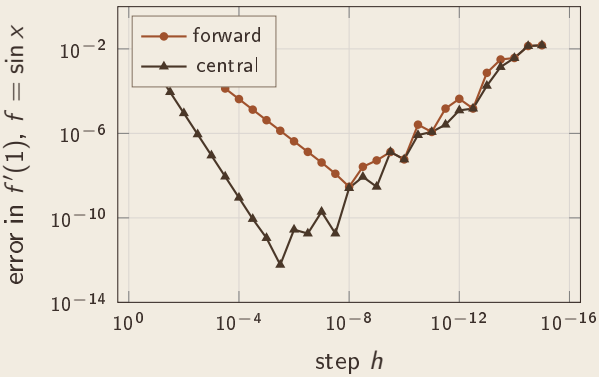
Finite differences

From the Taylor series $f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2}f''(x) \pm \frac{h^3}{6}f'''(x) + \ldots$:

Formula	**Leading error**	**Order**
Forward: $\dfrac{f(x+h) - f(x)}{h}$	$\frac{h}{2}f''(x)$	1
Central: $\dfrac{f(x+h) - f(x-h)}{2h}$	$\frac{h^2}{6}f'''(x)$	2

Example $f = \sin x$ at $x = 1$, $h = 0.1$: forward error 0.043, central error 0.0009. Central is the slope of the parabola through three neighbouring points.

Smaller h is not always better



Truncation error falls as h^p. Round-off error grows as ε/h, with $\varepsilon \approx 10^{-16}$ (Lecture 1).

	best h	best error
Forward	10^{-8}	3×10^{-9}
Central	3×10^{-6}	6×10^{-13}

Same U-curve as Lecture 2

Below the best h the answer gets **worse**: $f(x+h) - f(x)$ subtracts two nearly equal numbers.

The second derivative

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

error $\frac{h^2}{12} f''''(x)$: order 2

$f = \sin x$ at $x = 1$	
h	error
10^{-2}	7×10^{-6}
10^{-4}	3×10^{-9}
10^{-6}	8×10^{-5}

Round-off is worse

Round-off now grows as ε/h^2, so the best h is larger, about 10^{-4}.

This formula is the basis of solving differential equations such as the heat or Schrödinger equation on a grid.

Richardson extrapolation: extrapolating to $h \to 0$

If the error is Ch^p with known p (Lecture 2), two results $A(h)$ and $A(h/2)$ remove the leading error:

$$\boxed{A \approx \frac{2^p A(h/2) - A(h)}{2^p - 1}}$$

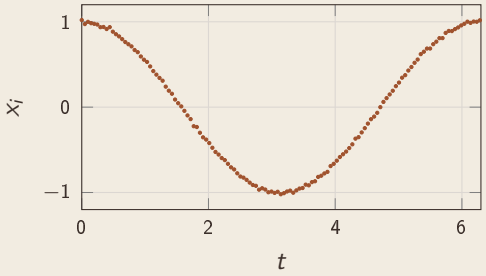
Central difference of $\sin x$ at $x = 1$ ($p = 2$); exact 0.5403023		
$A(0.1)$	0.5394023	error 9.0×10^{-4}
$A(0.05)$	0.5400772	error 2.3×10^{-4}
$\big(4A(0.05) - A(0.1)\big)/3$	0.5403022	error 1.1×10^{-7}

Why this extrapolation is safe

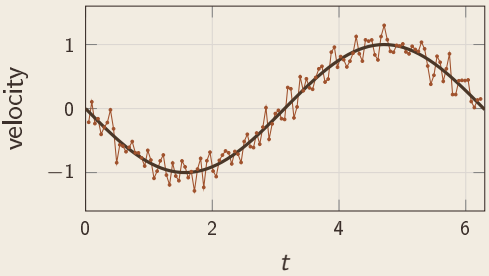
We extrapolate in h, where the form of the error is known — not in x, where it is not.

Measured-like data: differentiation amplifies noise

Samples: $x = \cos t$ plus noise



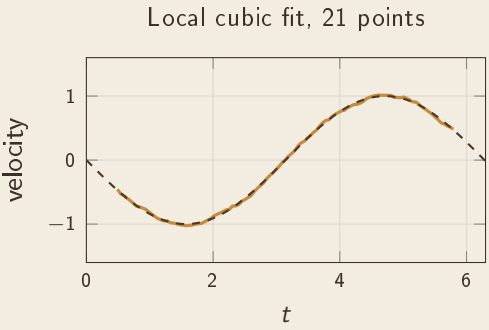
Central difference



Noise here: $x_i = \cos t_i + \varepsilon_i$, with measurement errors ε_i independent from sample to sample, mean 0, standard deviation $\sigma = 0.01$ (1% of the amplitude). Spacing $\Delta t = 0.049$.

Velocity error 0.15 (rms), against a truncation error of only 4×10^{-4}. For independent errors the noise in the derivative is $\sigma/(\sqrt{2}\,\Delta t)$: sampling 5 times faster makes it worse, 0.70.

Fit locally, then differentiate



At each point, fit a cubic to the 21 neighbouring samples by least squares (Lecture 3) and differentiate the cubic.

Velocity error 0.018 (rms): **8 times smaller**.

The trade-off

A wider window averages more noise but also smooths real variation. Keep the window shorter than the fastest time scale of the signal.

PART D

Numerical integration

Areas from values.

Trapezoid and Simpson

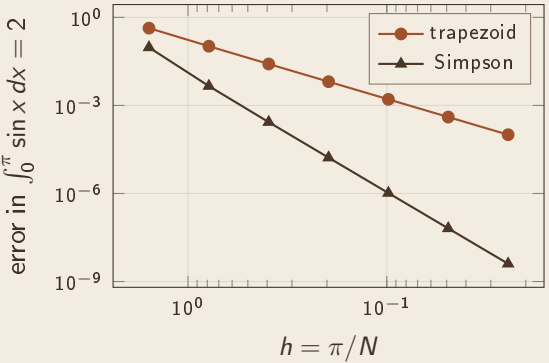
$\int_a^b f\,dx$ with N intervals of width $h = (b-a)/N$, values $f_0, \ldots, f_N$:

Rule (integrate the ...)	**Formula**	**Error**
Trapezoid (linear interpolant)	$h\left[\frac{1}{2}f_0 + f_1 + \cdots + f_{N-1} + \frac{1}{2}f_N\right]$	$O(h^2)$
Simpson (quadratic, N even)	$\frac{h}{3}\left[f_0 + 4f_1 + 2f_2 + 4f_3 + \cdots + 4f_{N-1} + f_N\right]$	$O(h^4)$

Both use only the values at the points, so they work directly on sampled data.

`np.trapezoid(f, x)`, `scipy.integrate.simpson(f, x=x)`

Error against the number of intervals



N	trapezoid	Simpson
4	0.10	4.6×10^{-3}
8	0.026	2.7×10^{-4}
16	0.0064	1.7×10^{-5}
32	0.0016	1.0×10^{-6}

Doubling N divides the error by 4 (trapezoid) and by 16 (Simpson): orders 2 and 4.

Familiar

The same straight lines on a log–log plot, and the same $h/2$ test, as for time stepping in Lecture 2.

When f can be evaluated anywhere: Gauss and adaptive

Gauss–Legendre quadrature

Choose the n points *and* their weights so that the rule is exact for polynomials up to degree $2n - 1$.

$\int_0^\pi \sin x \, dx$:
3 points: error 1.4×10^{-3}
5 points: error 1.1×10^{-7}
(Simpson, 5 points: 4.6×10^{-3})

Adaptive quadrature

Split the interval, estimate the error on each part, refine only where needed. Returns the integral **and** an error estimate.

```
val, err = scipy.integrate.quad(f, a, b)
```

Use these when f is a formula or a simulation you can call at any x. For sampled data, use trapezoid or Simpson.

Look at the integrand first

Problem	Example	Remedy
Infinite range	$\int_0^\infty e^{-x^2}\,dx$	Substitution, or `quad(f, 0, np.inf)`
Integrable singularity	$\int_0^1 x^{-1/2}dx$	Substitution $x = u^2$ removes it; or `quad`
Kink or jump	$\int_0^2 \lvert x-1\rvert\,dx$	Split the interval at the kink
Rapid oscillation	$\int_0^{10} \sin(100x)\,dx$	Several points per period, or special methods

Rule

The error estimates of all these methods assume a smooth integrand. Plot f before integrating it — as you plot data before fitting it.

Measured-like data: integration smooths fast fluctuations

Same samples as in Part C: $x = \cos t$, $\Delta t = 0.049$, noise $\sigma = 0.01$. Trapezoid rule on 33 samples for $\int_0^{\pi/2} x\,dt = 1$.

Source of error	size
Trapezoid (truncation)	2×10^{-4}
Measurement noise (std)	3×10^{-3}

For independent errors, noise in the integral $\approx \sigma\,\Delta t\sqrt{N}$: 0.3% of the result, though each sample has 1% noise.

The asymmetry

Differentiation divides differences of neighbours by Δt: rapid sample-to-sample changes are amplified.

Integration adds values: errors of opposite sign partly cancel, so rapid fluctuations are smoothed; errors of the same sign accumulate — an offset b grows as bT, independent errors as $\sigma\sqrt{\Delta t\,T}$.

Integration and the ODE solvers of Lecture 2

$\int_a^b f(t)\,dt$ is the solution $y(b)$ of $\dot{y} = f(t)$, $y(a) = 0$. Applied to this equation, the time-stepping methods become integration rules:

ODE method	Becomes the rule	Order
Euler	left rectangle: $h\,f(t_n)$ per step	1
RK2 (midpoint)	midpoint: $h\,f(t_n + \frac{h}{2})$	2
RK4	Simpson on $[t_n, t_n + h]$	4

The orders you measured in Lecture 2 are the orders of these integration rules.

Which method for which task?

Task	First choice	Error	Watch out for
Interpolation	cubic spline (linear if rough)	$O(h^4)$ ($O(h^2)$)	high-degree polynomials on equally spaced points
Extrapolation	a physical model, fitted	grows with distance	polynomials and splines; a system that changes
Differentiation	central difference	$O(h^2)$ + round-off	noise: fit locally first
Integration	Simpson (data), `quad` (formula)	$O(h^4)$ / estimated	singularities, kinks; offsets and drifts in data, which accumulate

Verify: halve h and compare, or leave out every second point.

Before you interpolate, differentiate or integrate

1. Are the values exact (formula, simulation) or measured with noise? *(interpolate, or fit first)*
2. Is f smooth on the scale of the spacing h? *(otherwise the orders do not hold)*
3. Am I inside the range of the points? *(if not, which model justifies extrapolating?)*
4. For a derivative: do noise or round-off dominate? *(choose h, or smooth first)*
5. Have I checked by halving h or by leaving out points? *(verification)*

Differentiation amplifies fast fluctuations;
integration smooths them but accumulates slow ones.

TO TAKE AWAY

Between the points we approximate; beyond them we guess.

Replace the values by a simple function, know its error Ch^p, check it by halving h — and remember: differentiation amplifies fast fluctuations; integration smooths them but accumulates slow ones.